University of South Florida

CDA 4205L Computer Architecture Lab

Lab Report

Semester: Fall 2026		
Experiment	*Number:*	6
	Date:	10/8/2026
Lab	*Section:*	003
	Lab TA:	Youjian Tang
Report	*Due Date:*	10/15/2026
Group ***(Lab3 onwards)***	*Member #1 name:*	Felix Hoang
	Member #2 name:	Jordan Martin

ABSTRACT:

The purpose of this lab was to design and simulate an Arithmetic Logic Unit (ALU) and a Control Unit (CU) for a simplified RISC-V processor. Using Verilog in EDA Playground with the Icarus Verilog simulator, we built the ALU to handle 14 different arithmetic, shift, bitwise, and comparison operations. We also created a CU to decode 7-bit opcodes into their corresponding datapath control signals. Both modules were tested using testbenches to verify their outputs, including experiments to see how Verilog handles signed and unsigned numbers. The testbenches successfully confirmed that both designs matched the expected behavior.

I. INTRODUCTION:

The ALU and control unit are core parts of every processor. The ALU does the actual math and logic operations that the instructions ask for, while the control unit decodes the opcodes to tell the rest of the processor how to route the data. In this lab, we transitioned from the software side of encoding assembly to the hardware side of executing it.

We used Verilog to build these two hardware components. We wrote the code for the ALU and CU using combinational logic and then used testbenches to simulate them and verify they worked correctly. This lab serves as the foundation for the single-cycle processors we will build later in the course.

II. METHODOLOGY:

Setup and Resources:

Software tools: EDA Playground (browser-based) with the Icarus Verilog 12.0 simulator, the Verilog HDL, a screenshot tool for capturing simulation output, and Microsoft Word for the report.

Submission files (all written by our group):

- **design.sv (in alu.zip):** Verilog design of the 32-bit signed ALU implementing the 14 operations in the operation table, with a default result of 0.
- **testbench.sv (in alu.zip):** Testbench that instantiates the ALU and applies each operation code to positive and mixed-sign operands, printing results with $monitor.
- **design.sv (in cu.zip):** Verilog design of the control unit that decodes the 7-bit opcode into eight 1-bit control signals and the 2-bit ALUOp.
- **testbench.sv (in cu.zip):** Testbench that applies all seven opcodes (R-type, Load, Jalr, I-type, S-type, B-type, J-type) to the CU and prints the outputs with $monitor.
- **CDA4205L_Lab6_Report.pdf:** This lab report with all screenshots and answers to T1 to T8.

Development Process:

We approached the lab as two independent but similar problems, an ALU and a CU, each built as a purely combinational module and each paired with its own testbench in a separate EDA Playground project, as the instructions required. Figure 1 shows the major components of each part and how the testbench connects to the design under test.

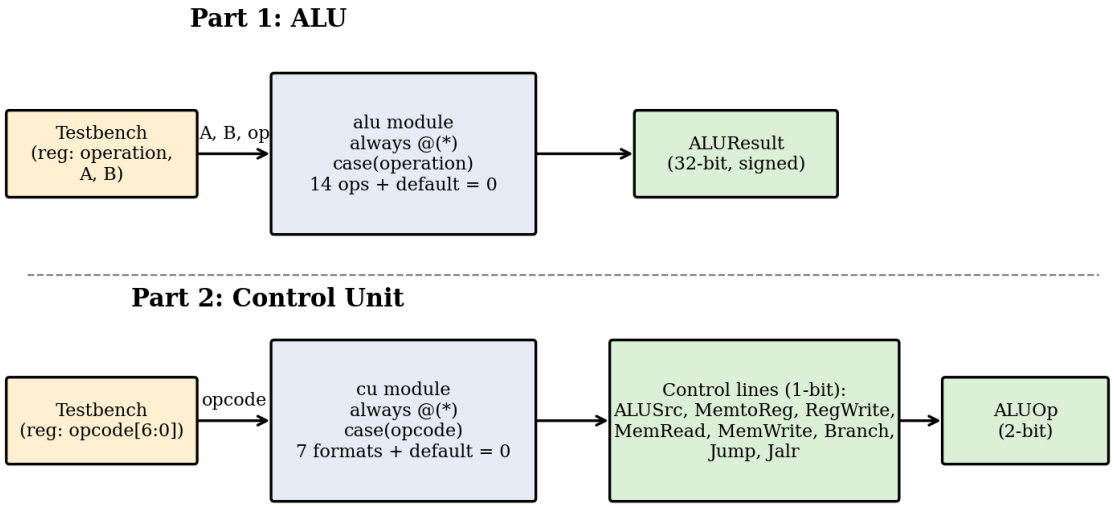


Figure 1: Block diagram of the ALU (Part 1) and CU (Part 2) designs and their testbenches.

ALU. We started by writing the module header exactly as specified (4-bit operation, two 32-bit signed operands, and a 32-bit signed output reg) with `timescale 1ns/1ns. We then built the logic with an always @(*) block containing a case statement and first implemented only the add case to confirm the skeleton compiled and simulated. After that, we added the remaining cases one category at a time (arithmetic, shift, bitwise, comparison) using the operation codes from the lab table, and set the default case to 0 so that undefined codes produce a known result. Comparison operators return 1 or 0, which Verilog extends to 32 bits. The finished design is shown in Listing 1.

Listing 1: ALU design (design.sv in alu.zip).

```
`timescale 1ns / 1ns
module alu(
  input  signed [3:0]  operation,
  input  signed [31:0] A,
  input  signed [31:0] B,
  output reg signed [31:0] ALUResult
);
  always @(*) begin
    case (operation)
      4'b0010: ALUResult = A + B;     // add
      4'b0110: ALUResult = A - B;     // sub
      4'b0111: ALUResult = A * B;     // mul
      4'b1111: ALUResult = A / B;     // div
      4'b0100: ALUResult = A << B;    // sll
      4'b1100: ALUResult = A >> B;    // srl
      4'b1110: ALUResult = A >>> B;   // sra
      4'b0000: ALUResult = A & B;     // and
      4'b0001: ALUResult = A | B;     // or
      4'b0011: ALUResult = A ^ B;     // xor
```

```
    4'b1000: ALUResult = (A == B);  // eq
    4'b1001: ALUResult = (A != B);  // neq
    4'b1010: ALUResult = (A <  B);  // lt
    4'b1011: ALUResult = (A >= B);  // geq
    default: ALUResult = 0;
   endcase
 end
endmodule
```

CU. The control unit has a 7-bit unsigned opcode input and nine outputs (eight 1-bit control signals and the 2-bit ALUOp), all declared as output reg. Because each case assigns nine signals, every case is wrapped in a begin/end block. We transcribed each row of the CU control table into one case, using the opcodes 0110011 (R-type), 0000011 (Load), 1100111 (Jalr), 0010011 (I-type), 0100011 (S-type), 1100011 (B-type), and 1101111 (J-type), and added a default case that drives every control line to 0 so an unrecognized opcode cannot accidentally enable a memory or register write. Listing 2 shows an excerpt of the design.

Listing 2: CU design excerpt (design.sv in cu.zip).

```
module cu(
 input [6:0] opcode,
 output reg ALUSrc, MemtoReg, RegWrite, MemRead,
 output reg MemWrite, Branch, Jump, Jalr,
 output reg [1:0] ALUOp
);
 always @(*) begin
   case (opcode)
     7'b0110011: begin // R-type
       ALUSrc=0; MemtoReg=0; RegWrite=1; MemRead=0;
       MemWrite=0; Branch=0; Jump=0; Jalr=0; ALUOp=2'b10;
     end
     // ... Load, Jalr, I-type, S-type, B-type, J-type follow the same pattern ...
     default: begin
       ALUSrc=0; MemtoReg=0; RegWrite=0; MemRead=0;
       MemWrite=0; Branch=0; Jump=0; Jalr=0; ALUOp=2'b00;
     end
   endcase
 end
endmodule
```

Testbenches. Each testbench declares a reg for every design input and a wire for every design output, instantiates the unit under test, and uses an initial block with a $monitor statement so that every change is printed automatically. In the ALU testbench we set A and B, then used #5 delays between operation codes so that each operation gets its own line of output. In the CU testbench we stepped through all seven opcodes the same way. After the positive-operand run (T1) we changed B to a negative value (T2), and then removed the signed keyword from A (T3) and then from B (T4) to study how Verilog handles mixed signed/unsigned expressions. Finally, we enabled "Download files after run" to export alu.zip and cu.zip.

III. RESULTS:

All results below were produced in EDA Playground with Icarus Verilog 12.0. The expected values in each table were computed by hand following Verilog's evaluation rules and are used to confirm each screenshot.

T1: Take a screenshot of your output when A and B are both positive values. Confirm that the results are correct.

For this test, we set A = 197 and B = 237. The ALU output perfectly matched our expected hand calculations for all 14 operations. For example, the add operation gave 434. The division and shift operations output 0 because integer division truncates and the shift amount (237) is larger than the 32-bit limit.

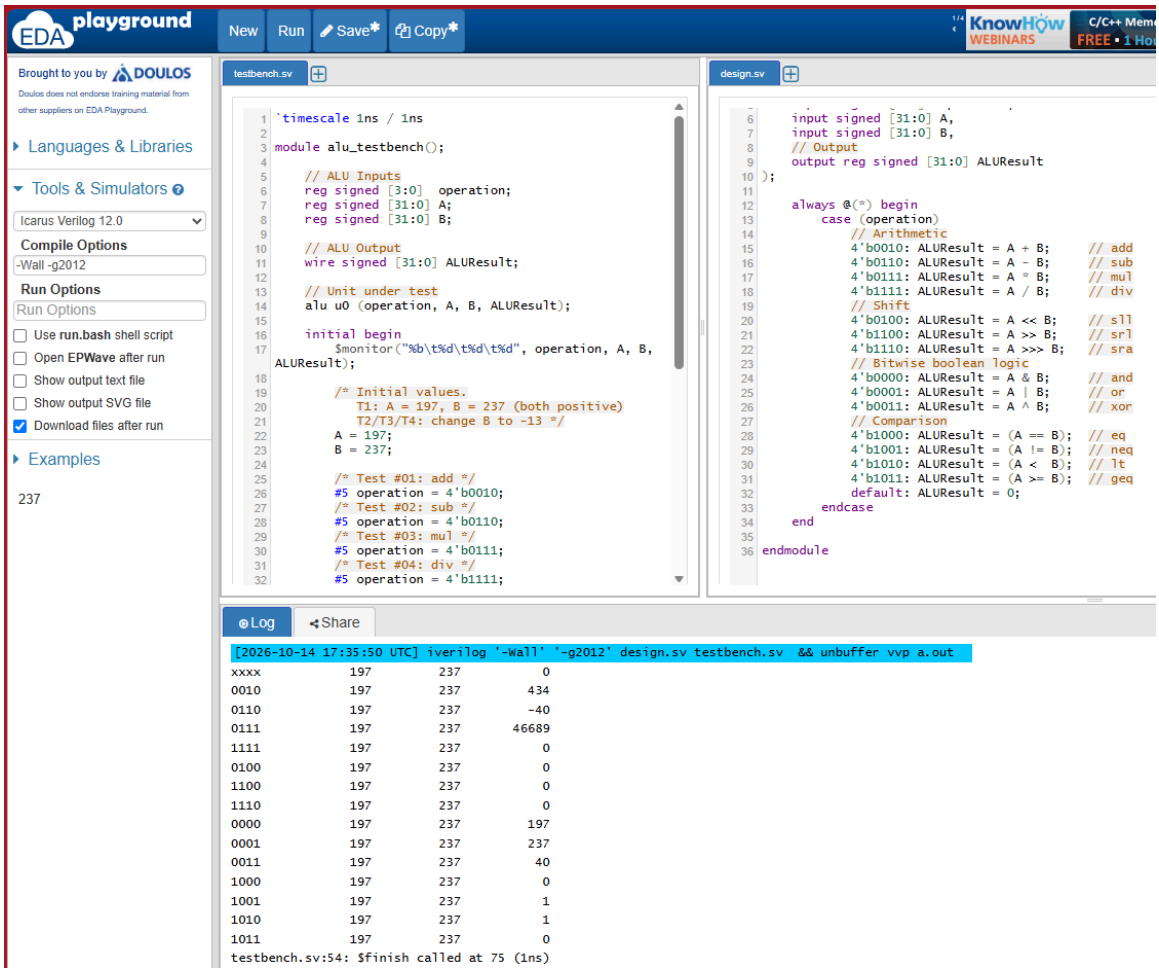


Figure 2: ALU simulation output with A = 197 and B = 237 (T1).

Operation	Name	Expected ALUResult (A=197, B=237)
0010	add	434
0110	sub	-40
0111	mul	46689
1111	div	0
0100	sll	0
1100	srl	0
1110	sra	0
0000	and	197

0001	or	237
0011	xor	40
1000	eq	0
1001	neq	1
1010	lt	1
1011	geq	0

Table 1: Expected ALU results for T1.

T2: Take a screenshot of your output when A is positive and B is negative. Confirm that the results are correct.

We kept A = 197 and changed B to -13. The ALU correctly handled the two's complement arithmetic. Multiplication gave -2561 and division gave -15. The bitwise and shift operations also gave the expected outputs based on the binary representation of a negative number.

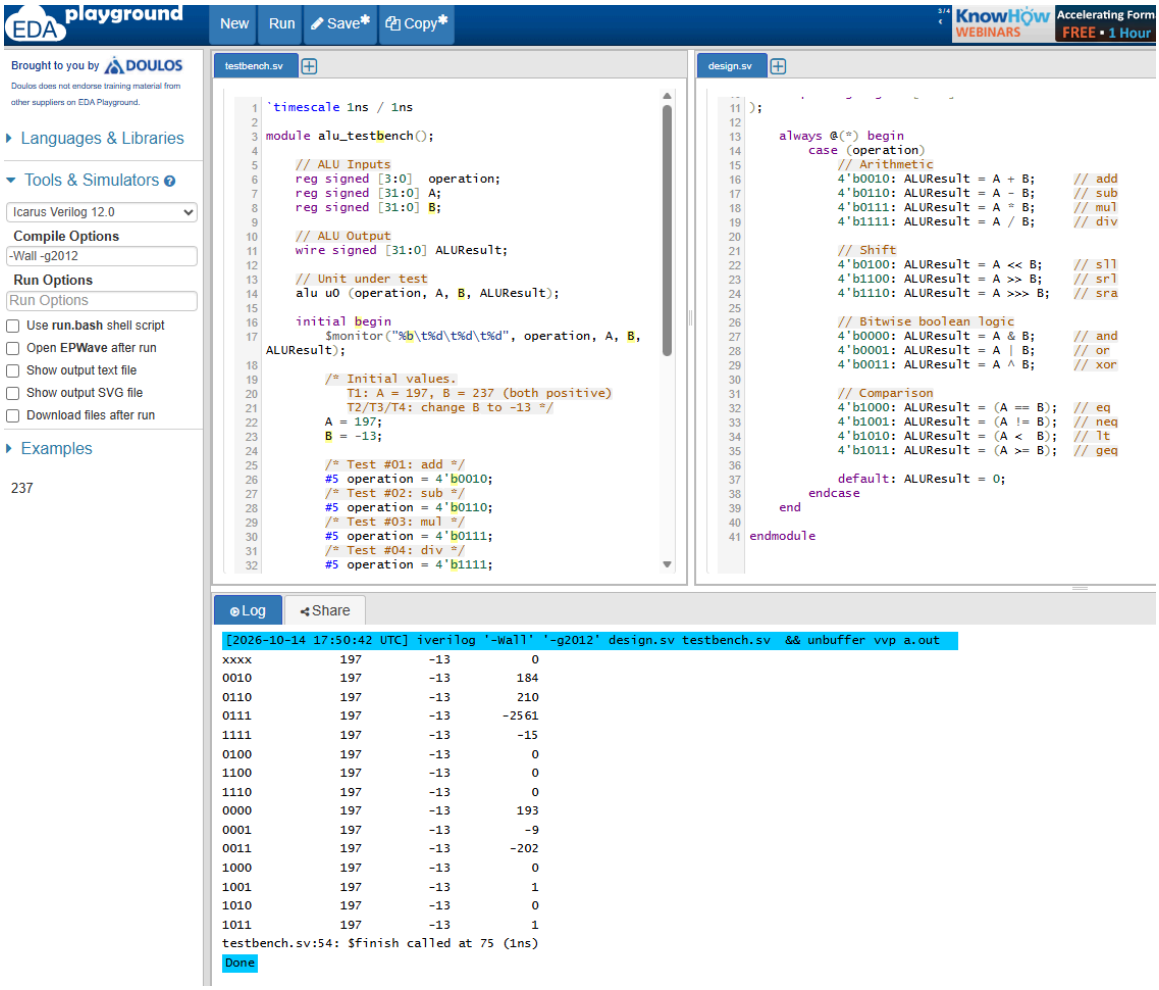


Figure 3: ALU simulation output with A = 197 and B = −13 (T2).

Operation	Name	Expected ALUResult (A=197, B=-13)
0010	add	184
0110	sub	210
0111	mul	-2561

1111	div	-15
0100	sll	0
1100	srl	0
1110	sra	0
0000	and	193
0001	or	-9
0011	xor	-202
1000	eq	0
1001	neq	1
1010	lt	0
1011	geq	1

Table 2: Expected ALU results for T2.

T3: Remove the signed modifier from A in the ALU module and rerun using the same A and B values as in T2. Take a screenshot of your output and explain what happens.

After we removed the signed keyword from A, the basic math (add, sub, mul) and bitwise operations stayed the same. However, Verilog started treating the entire expression as unsigned. Because of this, B (-13) was read as a massive positive number (4,294,967,283), which completely changed the answers for division and the less-than/greater-than comparisons.

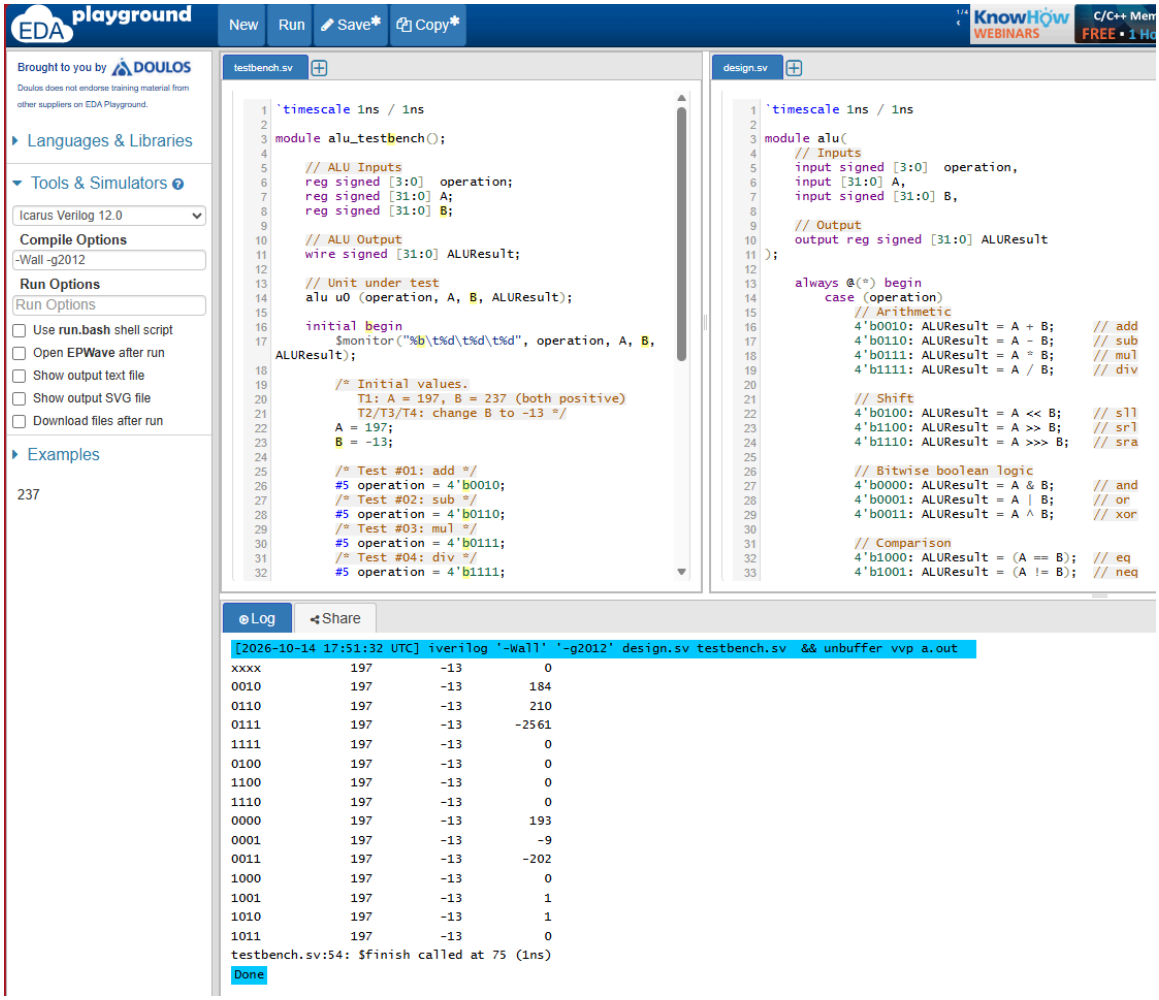


Figure 4: ALU simulation output with the signed modifier removed from A (T3).

Name	T2 (both signed)	T3 (A unsigned)
div	-15	0
lt	0	1
geq	1	0
All other operations	Same	Same

Table 3: Operations whose results change when A is unsigned (A = 197, B = −13).

T4: Remove the signed modifier from B in the ALU module and rerun using the same A and B values as in T2. Take a screenshot of your output and explain what happens.

Taking the signed keyword off of B instead of A gave the exact same incorrect results as T3. If just one operand is unsigned, Verilog treats the whole expression as unsigned. This proved that we need to keep both inputs explicitly declared as signed in the ALU.

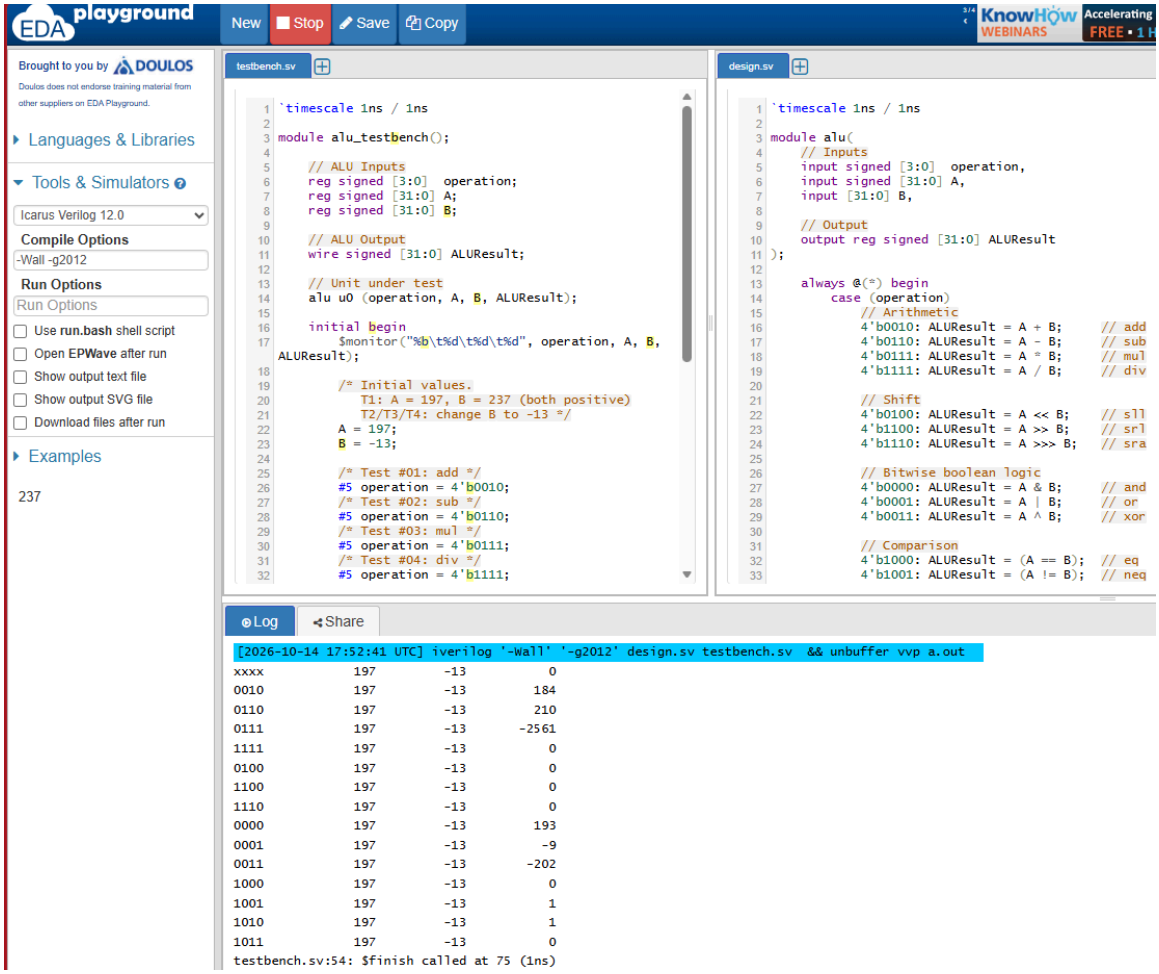


Figure 5: ALU simulation output with the signed modifier removed from B (T4).

T5: Take a screenshot of your output with all "opcodes". Confirm that the results are correct.

We ran the CU testbench to check all seven RISC-V instruction formats. The monitor output matched the control table from the lab handout perfectly, setting the right 1s and 0s for signals like RegWrite, ALUSrc, and ALUOp based on each opcode.

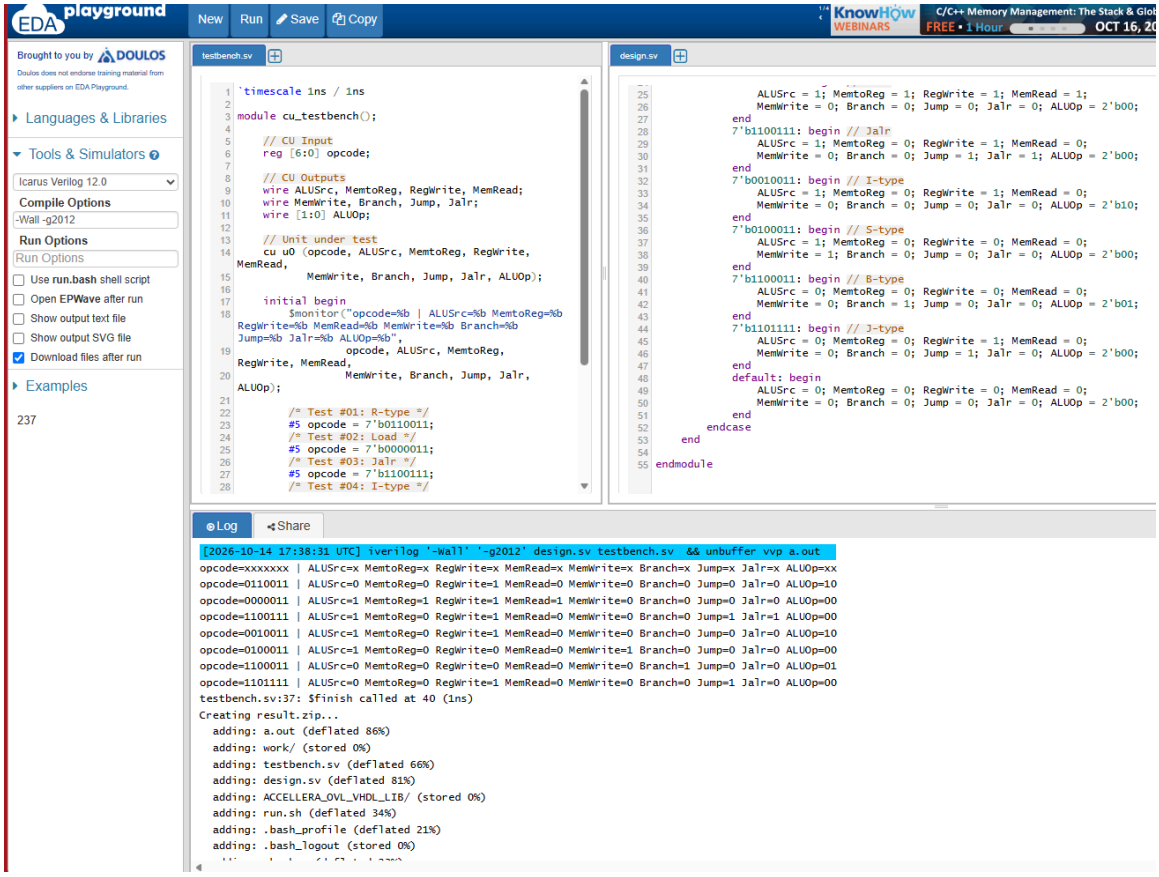


Figure 6: CU simulation output for all opcodes (T5).

Format	Opcode	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	Jump	Jalr	ALUOp
R-type	0110011	0	0	1	0	0	0	0	0	10
Load	0000011	1	1	1	1	0	0	0	0	00
Jalr	1100111	1	0	1	0	0	0	1	1	00
I-type	0010011	1	0	1	0	0	0	0	0	10
S-type	0100011	1	0	0	0	1	0	0	0	00
B-type	1100011	0	0	0	0	0	1	0	0	01
J-type	1101111	0	0	1	0	0	0	1	0	00

Table 4: Expected CU control-line values for each opcode.

T6: The #delay specified in step 13 was 5 "time units". What is the real value of these 5 "time units" in the simulation?

The real value is 5 nanoseconds (ns). We used the'timescale 1ns / 1ns directive at the start of our code, so every #5 delay in the testbench moves the simulation forward by 5 ns.

T7: What is the purpose of a testbench? Why is it needed for each module (ALU and CU)?

A testbench lets us simulate and verify a Verilog module in software before putting it on actual hardware. It supplies input values and prints the outputs. We needed separate testbenches for the ALU and CU because they are completely different modules with different inputs, outputs, and expected behaviors.

T8: Can we obtain power, performance, area (PPA) results with the testbench? Why?
No, you cannot get PPA results from this testbench. This simulation only checks if the logic and math are correct. To get PPA numbers, you have to synthesize the design onto real hardware or an FPGA, which requires physical implementation tools instead of just a behavioral simulator.

IV. DISCUSSION:

The results demonstrate that our Verilog code correctly implemented the ALU and CU for a basic RISC-V processor. The testbenches verified that our case statements properly selected the right math operations and control lines. One of the main things we learned was how Verilog handles signed and unsigned numbers, as seen in T3 and T4. If a single signed keyword is missing, operations that rely on the sign bit like division and relational operators will give completely wrong answers.

V. CONCLUSION

This lab successfully demonstrated how to design and simulate a 32-bit ALU and a 7-bit Control Unit. By writing the hardware modules and their testbenches in Verilog, we verified that the components can decode instructions and perform the correct operations. We also learned how Verilog's signed and unsigned expression rules affect hardware design. The successful simulations confirm that our modules are ready to be integrated into a larger processor datapath in future labs.

REFERENCES:

[1] "CDA 4205L Lab #6: ALU & Control Unit Design," University of South Florida.

[2] "alu.zip" and "cu.zip," Lab 6 Verilog designs and testbenches.